

Parameterized Results for Interval Scheduling with Machine Eligibility

Danny Hermelin, Yuval Itzhaki, Dvir Shabtay, Hendrik Molter

- In **Just-in-Time** scheduling jobs are desirably completed *exactly* on their deadlines.



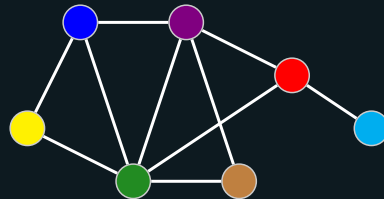
- In **Just-in-Time** scheduling jobs are desirably completed *exactly* on their deadlines.
- Effectively, every job has a predetermined execution interval.



Just-in-Time Motivation

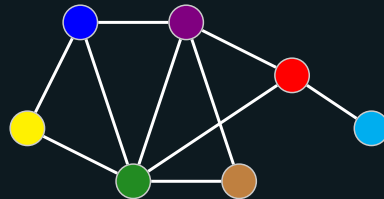


- In **Just-in-Time** scheduling jobs are desirably completed *exactly* on their deadlines.
- Effectively, every job has a predetermined execution interval.
- Jobs whose intervals intersect are in *conflict*.





- In **Just-in-Time** scheduling jobs are desirably completed *exactly* on their deadlines.
- Effectively, every job has a predetermined execution interval.
- Jobs whose intervals intersect are in *conflict*.
- The conflict-graph is an **interval graph**.

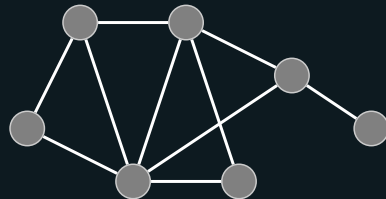


Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.





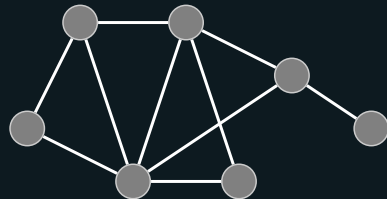
Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.

This problem is solvable in polynomial time!



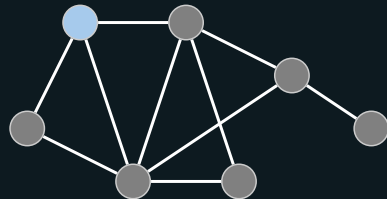
Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.

This problem is solvable in polynomial time!



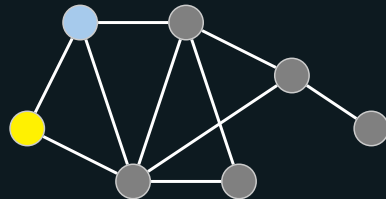
Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.

This problem is solvable in polynomial time!



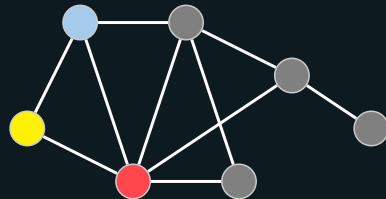
Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.

This problem is solvable in polynomial time!



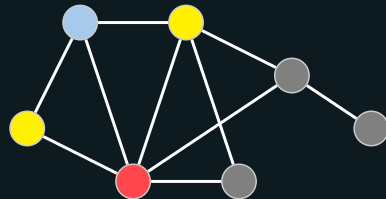
Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.

This problem is solvable in polynomial time!



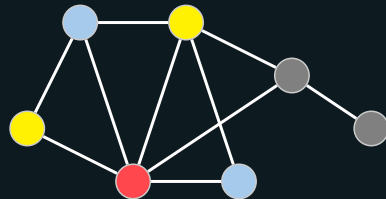
Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.

This problem is solvable in polynomial time!



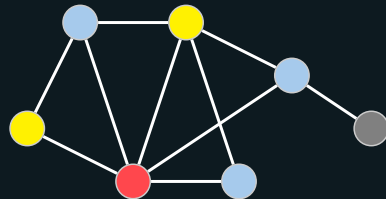
Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.

This problem is solvable in polynomial time!





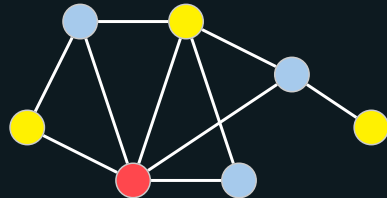
Interval Scheduling on Identical Machines

Input: m machines and n jobs, each job has

- a processing time p_j ,
- a weight w_j ,
- a deadline d_j

Output: Maximum weighted JIT schedule.

This problem is solvable in polynomial time!

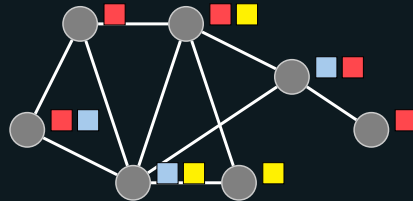


Interval Scheduling on **Eligible** Machines

Input: m machines and n jobs, each with:

- processing time p_j
- weight w_j
- deadline d_j
- **eligible machine set** $M_j \subseteq [m]$

Output: Maximum weighted JIT schedule.



Related Work



- The problem is *strongly* NP-hard [AS87].

- The problem is *strongly* NP-hard [AS87].
 - No $(Wn)^{O(1)}$ algorithm.

- The problem is *strongly* NP-hard [AS87].
 - No $(Wn)^{O(1)}$ algorithm.
- The problem is solvable in $\mathcal{O}(n^{m+1})$ [AS87].

- The problem is *strongly* NP-hard [AS87].
 - No $(Wn)^{O(1)}$ algorithm.
- The problem is solvable in $\mathcal{O}(n^{m+1})$ [AS87].
- Is the problem FPT with respect to m ? [MvB18].



Computers & Operations Research

Volume 100, December 2018, Pages 254-261



Survey in Operations Research and Management Science

Parameterized complexity of machine scheduling: 15 open problems

Matthias Mnich^{a, b}✉, René van Bevern^{c, d}✉

Arkin and Silverberg (1987) show that this problem is solvable in $O(n^2 \log n)$ time. However, they showed that the variant $P|M_j, d_j - r_j = p_j | \sum w_j U_j$ is NP-hard and solvable in $O(n^{m+1})$ time.

Open Problem 8

Is $P|M_j, d_j - r_j = p_j | \sum w_j U_j$ fixed-parameter tractable parameterized by the number of machines?

Theorem 1

The problem is strongly $W[1]$ -hard when parameterized by m , the number of machines.

Theorem 1

The problem is strongly $W[1]$ -hard when parameterized by m , the number of machines.

Theorem 2

The problem is NP-hard for $p_{\max} = \mathcal{O}(1)$, i.e. the longest processing time is constant.

Theorem 1

The problem is strongly $W[1]$ -hard when parameterized by m , the number of machines.

Theorem 2

The problem is NP-hard for $p_{\max} = \mathcal{O}(1)$, i.e. the longest processing time is constant.

Theorem 3

The problem is FPT with respect to $p_{\max} + m$.

Theorem 1

The problem is strongly $W[1]$ -hard when parameterized by m , the number of machines.

Theorem 2

The problem is NP-hard for $p_{\max} = \mathcal{O}(1)$, i.e. the longest processing time is constant.

Theorem 3

The problem is FPT with respect to $p_{\max} + m$.

Included in the 17th International Symposium on Parameterized and Exact Computation (IPEC 2022)

Published in Journal of Computer and System Sciences (JCSS-D-23-00304)

Theorem 1

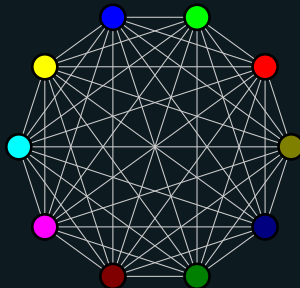
The problem is strongly $W[1]$ -hard when parameterized by m , the number of machines.



Theorem 1

The problem is strongly $W[1]$ -hard when parameterized by m , the number of machines.

- Parameterized reduction from MULTICOLORED K -CLIQUE

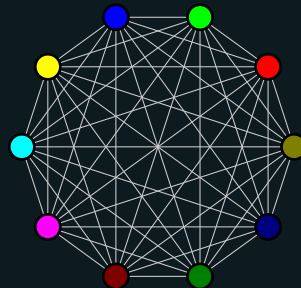




Theorem 1

The problem is strongly $W[1]$ -hard when parameterized by m , the number of machines.

- Parameterized reduction from MULTICOLORED K -CLIQUE
- Reduce such that $m \leq f(k)$

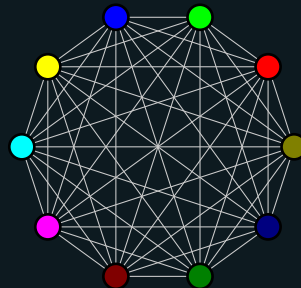




Theorem 1

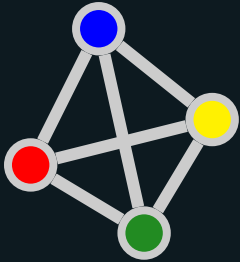
The problem is strongly $W[1]$ -hard when parameterized by m , the number of machines.

- Parameterized reduction from MULTICOLORED K-CLIQUE
- Reduce such that $m \leq f(k)$
- Key ideas:
 - Map all colorful edges and vertices to jobs
 - Schedule maximum vertices and edges jobs
 - Prevent the simultaneous schedule of non-adjacent vertices



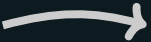


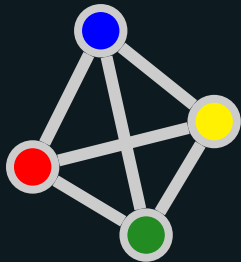
k -Multicolored Clique



Reduction from k -Multicolored Clique

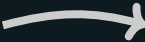


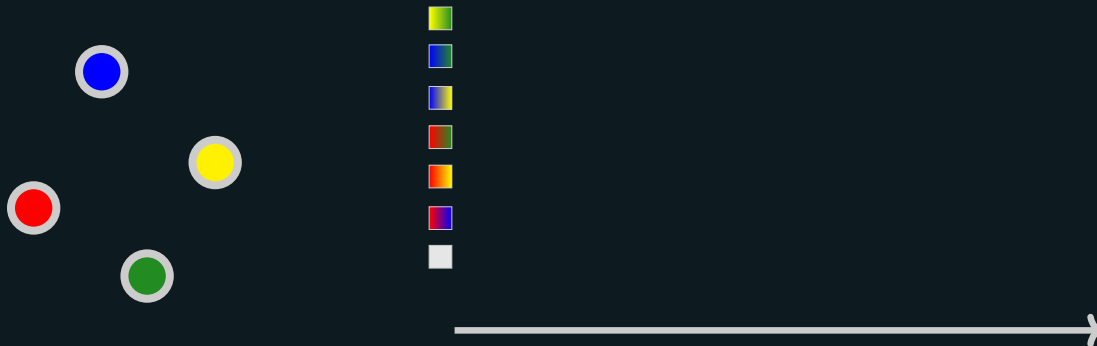
k -Multicolored Clique  Interval Scheduling with Eligible Machines



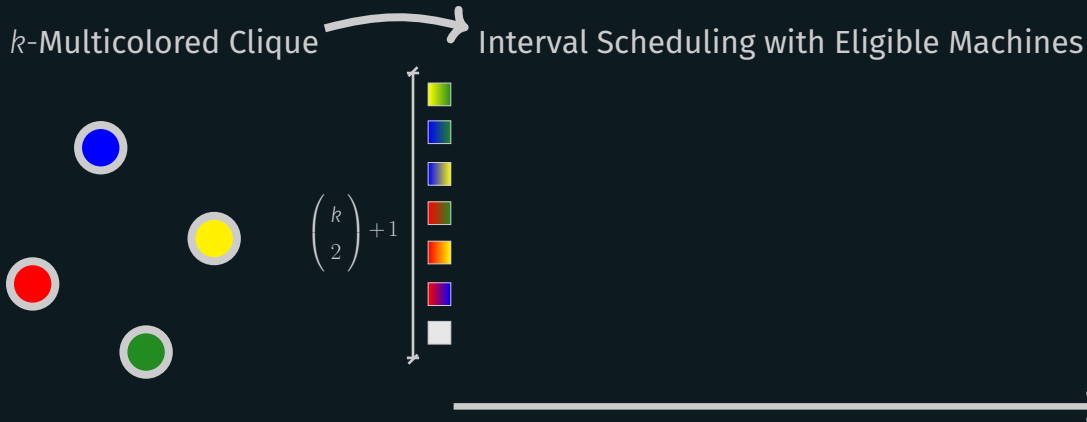
Reduction from k-Multicolored Clique



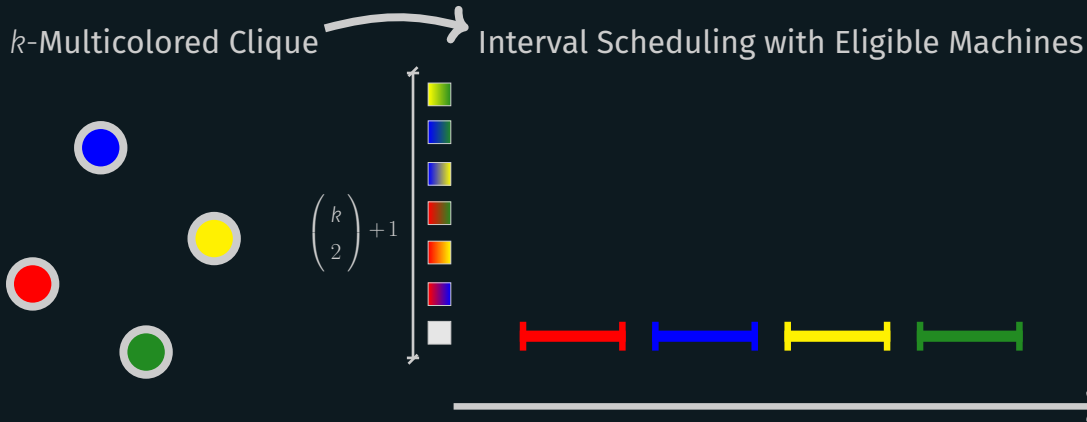
k -Multicolored Clique  Interval Scheduling with Eligible Machines



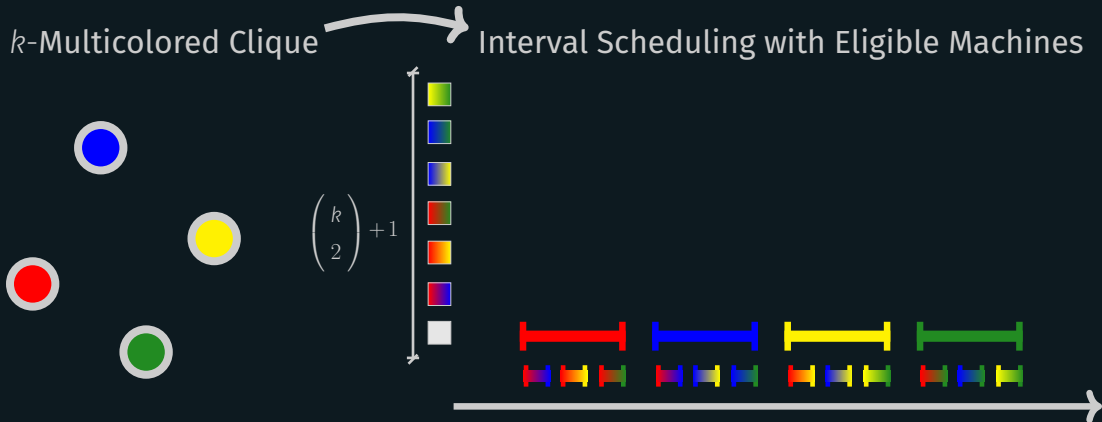
Reduction from k-Multicolored Clique



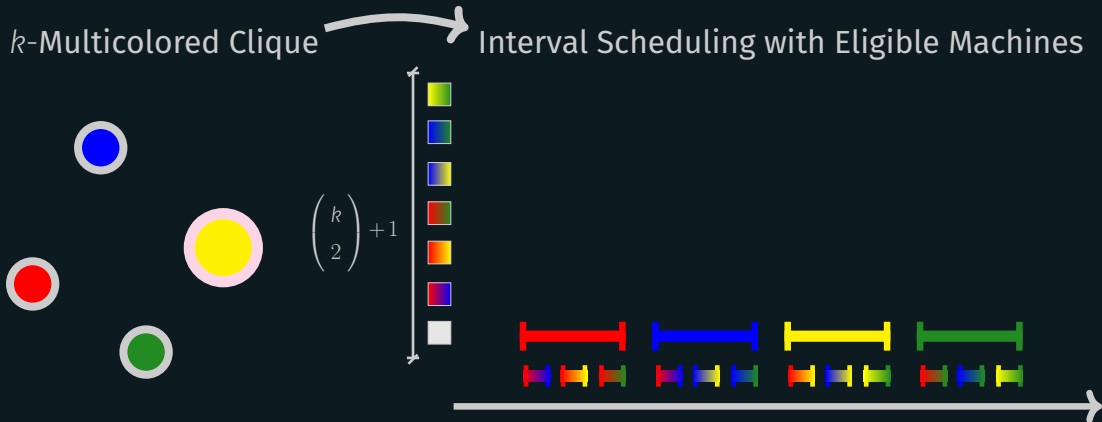
Reduction from k-Multicolored Clique



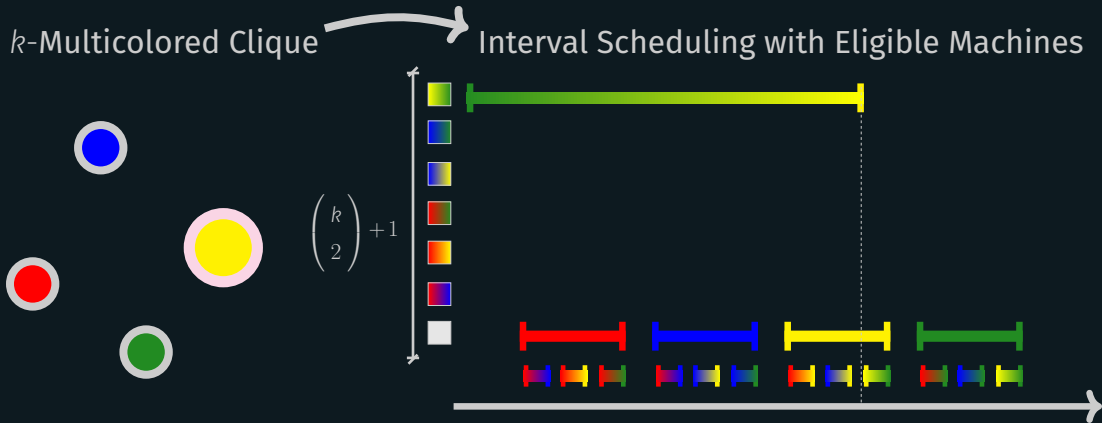
Reduction from k-Multicolored Clique



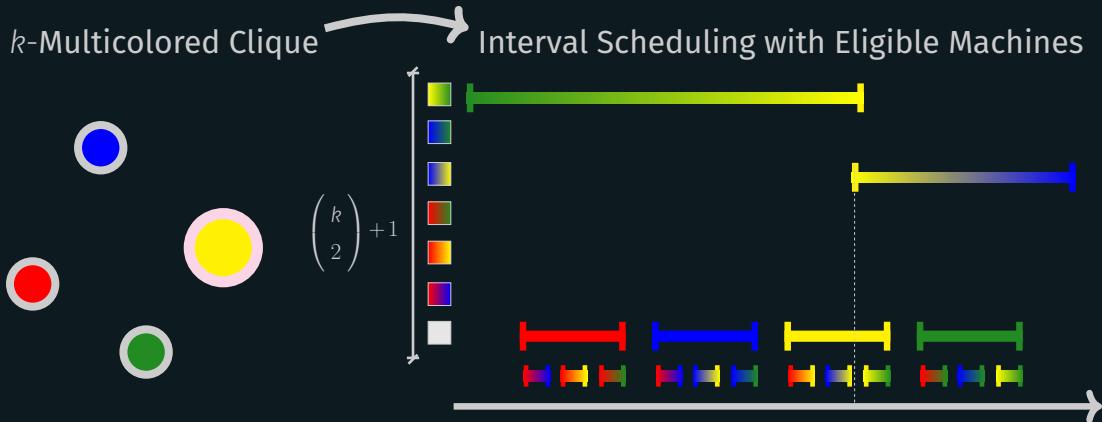
Reduction from k-Multicolored Clique



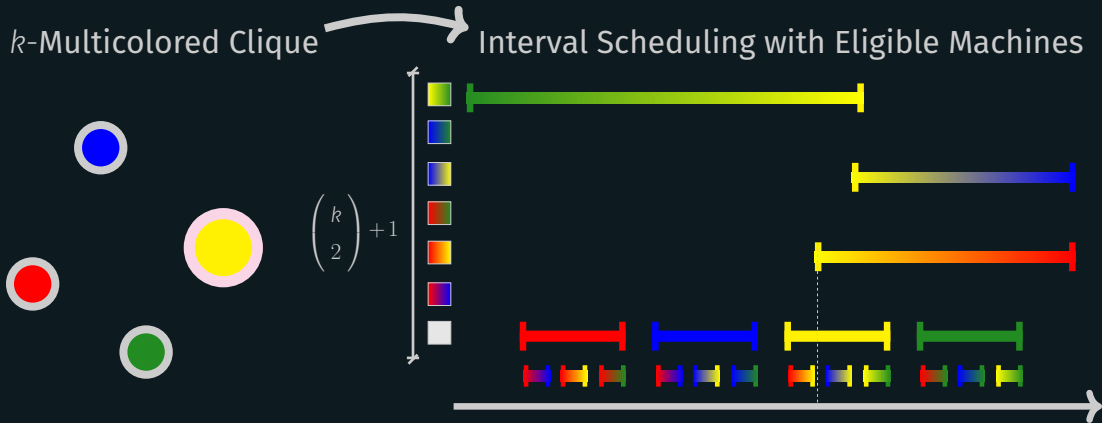
Reduction from k-Multicolored Clique



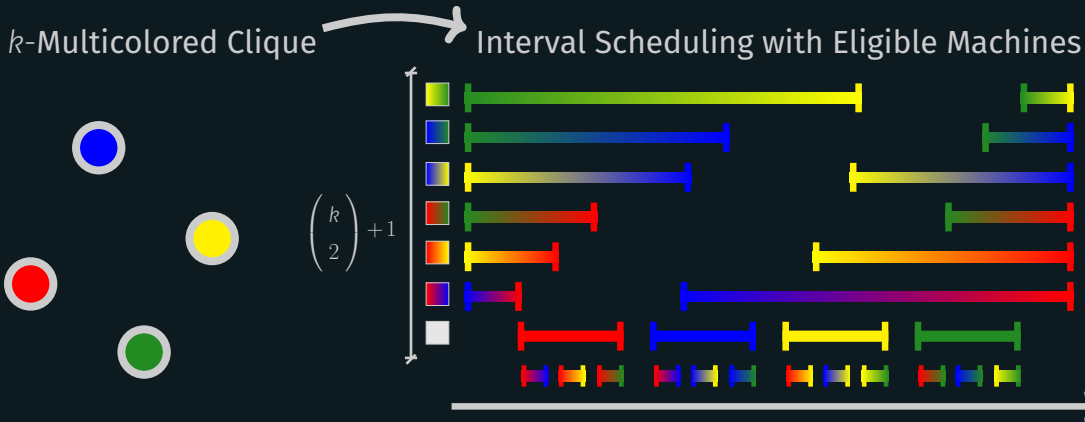
Reduction from k-Multicolored Clique



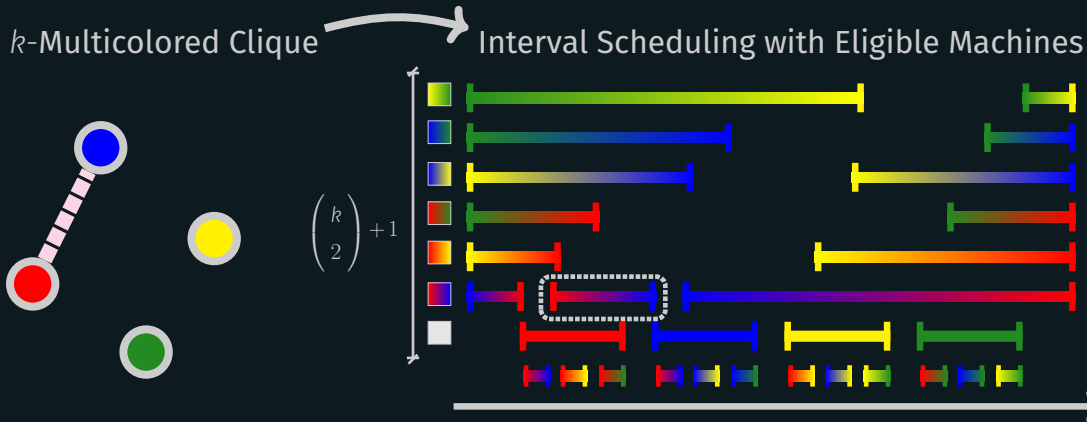
Reduction from k-Multicolored Clique



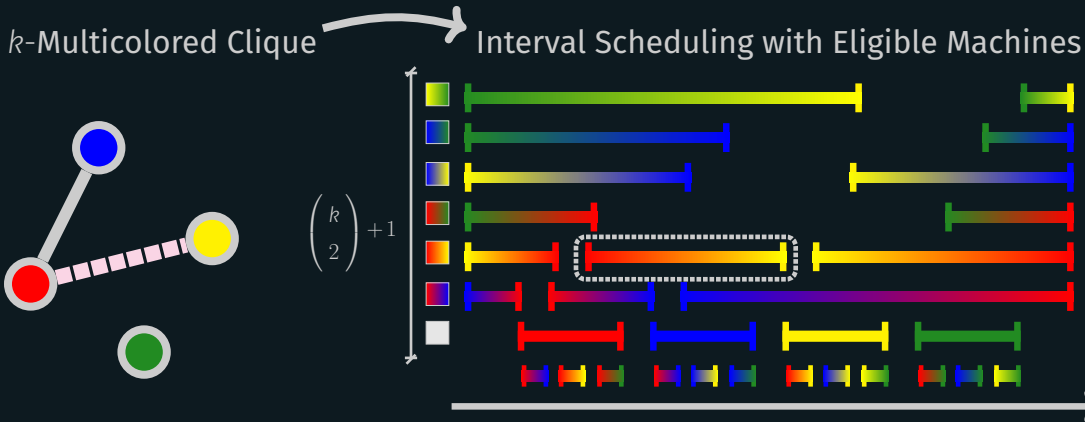
Reduction from k-Multicolored Clique



Reduction from k-Multicolored Clique



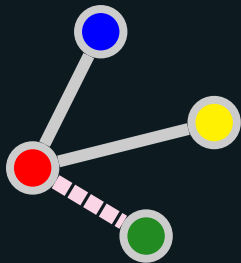
Reduction from k-Multicolored Clique



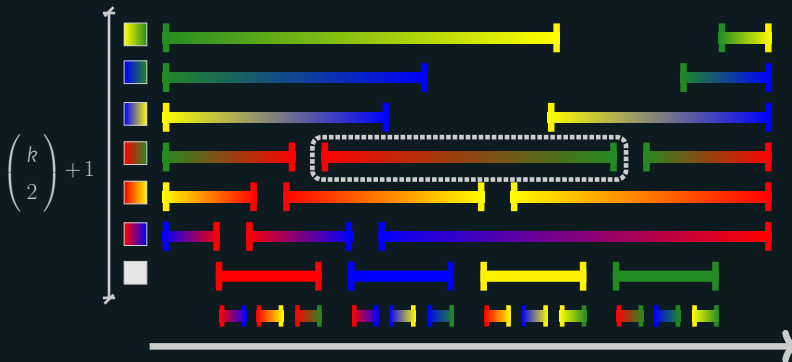
Reduction from k-Multicolored Clique



k -Multicolored Clique



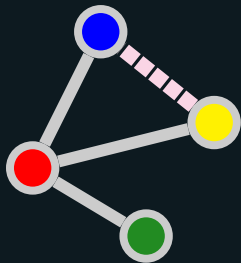
Interval Scheduling with Eligible Machines



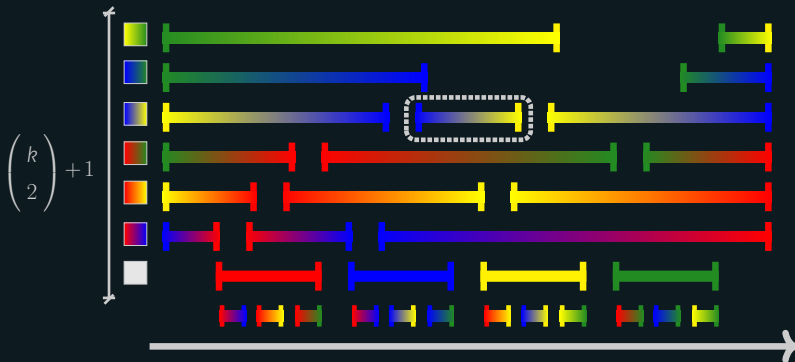
Reduction from k-Multicolored Clique



k -Multicolored Clique



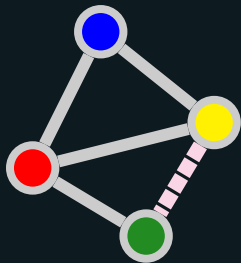
Interval Scheduling with Eligible Machines



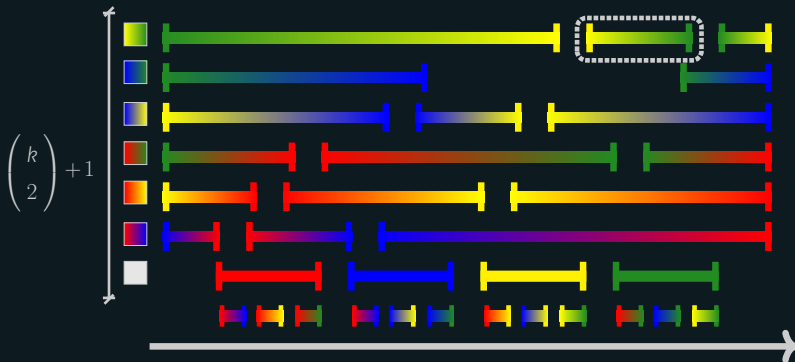
Reduction from k-Multicolored Clique



k -Multicolored Clique



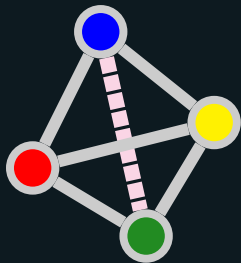
Interval Scheduling with Eligible Machines



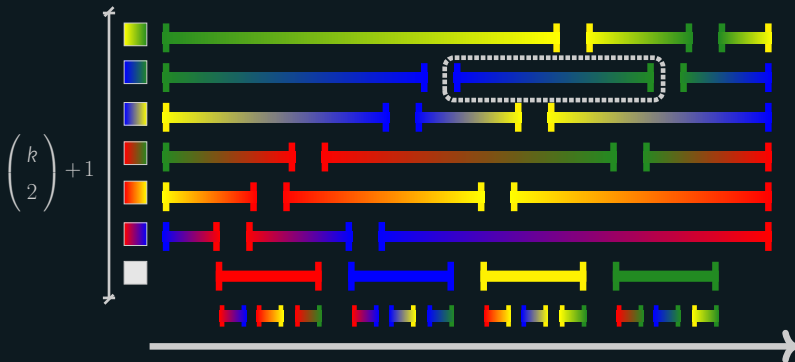
Reduction from k-Multicolored Clique



k -Multicolored Clique



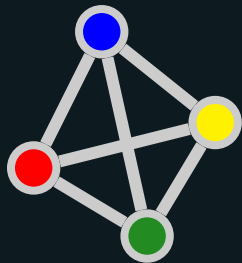
Interval Scheduling with Eligible Machines



Reduction from k-Multicolored Clique



k -Multicolored Clique



Interval Scheduling with Eligible Machines



Theorem 2

The problem is NP-hard for $p_{\max} = \mathcal{O}(1)$, i.e. the longest processing time is constant.

- Reduction from EXACT (3,4)-SAT
- Key ideas:

Theorem 2

The problem is NP-hard for $p_{\max} = \mathcal{O}(1)$, i.e. the longest processing time is constant.

- Reduction from EXACT (3,4)-SAT
- Key ideas:
 - Map all variables and literals to jobs
 - Schedule for every clause exactly one literal job
 - Scheduling literal jobs requires consistent variable jobs schedule
 - Jobs' processing times are bounded by a constant

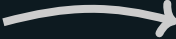
Reduction from Exact (3,4)-SAT



Exact (3,4)-SAT

Reduction from Exact (3,4)-SAT

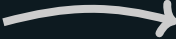


Exact (3,4)-SAT 

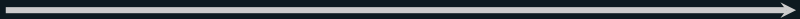
$$(x \vee y \vee \neg z)$$

Reduction from Exact (3,4)-SAT



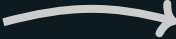
Exact (3,4)-SAT  Interval Scheduling with Eligible Machines

$$(x \vee y \vee \neg z)$$

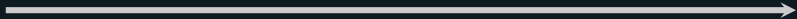


Reduction from Exact (3,4)-SAT



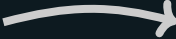
Exact (3,4)-SAT  Interval Scheduling with Eligible Machines

$$(x \vee y \vee \neg z)$$



Reduction from Exact (3,4)-SAT

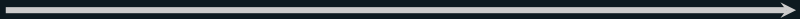


Exact (3,4)-SAT  Interval Scheduling with Eligible Machines

$$(x \vee y \vee \neg z)$$

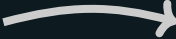
MXF 

MXT 



Reduction from Exact (3,4)-SAT



Exact (3,4)-SAT  Interval Scheduling with Eligible Machines

$$(x \vee y \vee \neg z)$$

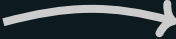
MXF 

MXT 



Reduction from Exact (3,4)-SAT



Exact (3,4)-SAT  Interval Scheduling with Eligible Machines

$$(x \vee y \vee \neg z)$$

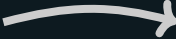
MYF 

MYT 



Reduction from Exact (3,4)-SAT

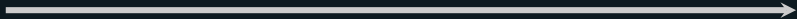


Exact (3,4)-SAT  Interval Scheduling with Eligible Machines

$$(x \vee y \vee \neg z)$$

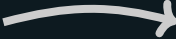
MZF 

MZT 



Reduction from Exact (3,4)-SAT



Exact (3,4)-SAT  Interval Scheduling with Eligible Machines

$$(x \vee y \vee \neg z)$$

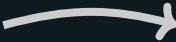
MXF 

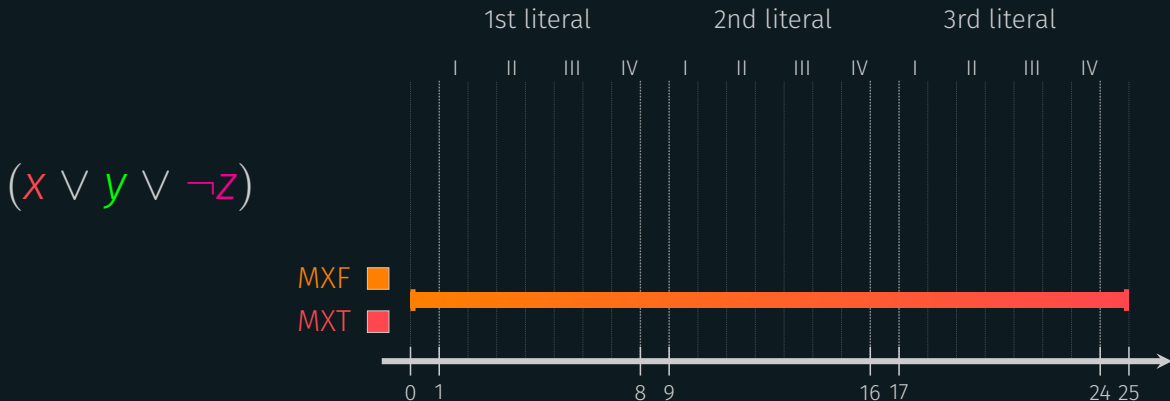
MXT 



Reduction from Exact (3,4)-SAT



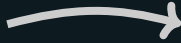
Exact (3,4)-SAT  Interval Scheduling with Eligible Machines



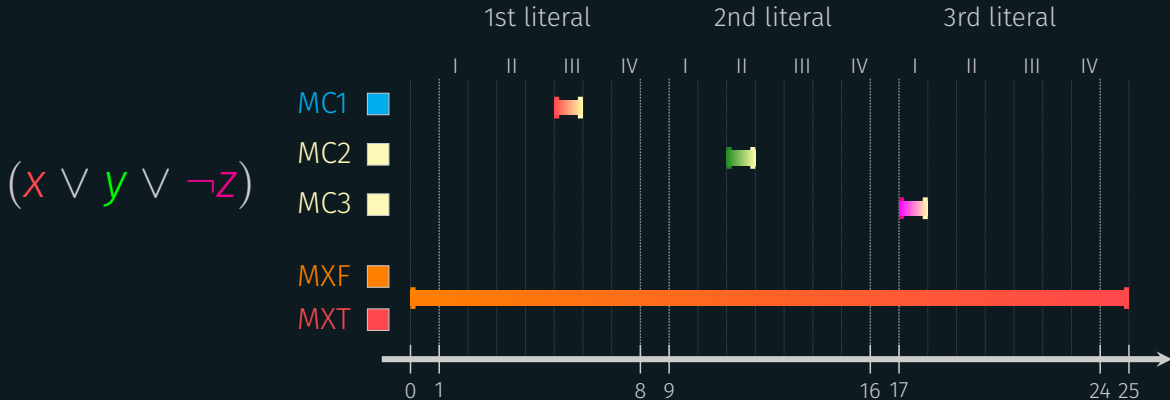
Reduction from Exact (3,4)-SAT



Exact (3,4)-SAT



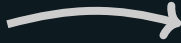
Interval Scheduling with Eligible Machines



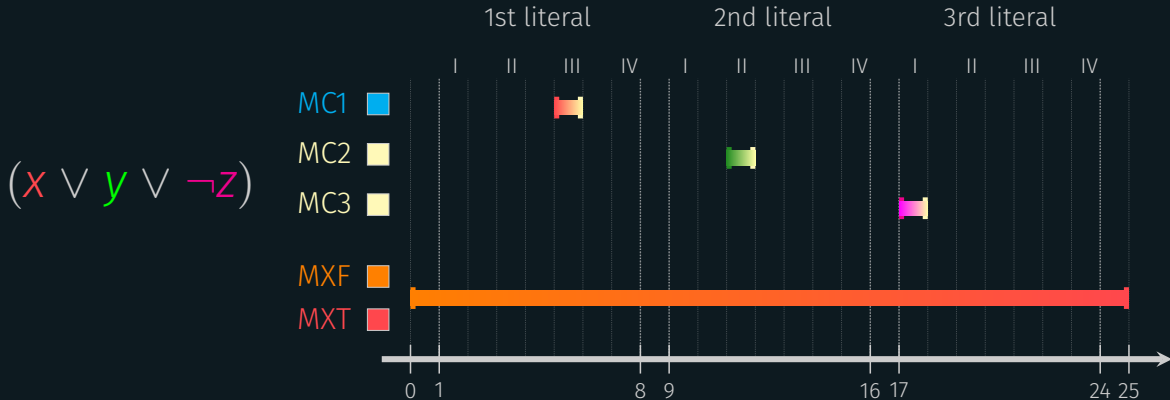
Reduction from Exact (3,4)-SAT



Exact (3,4)-SAT

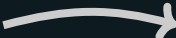


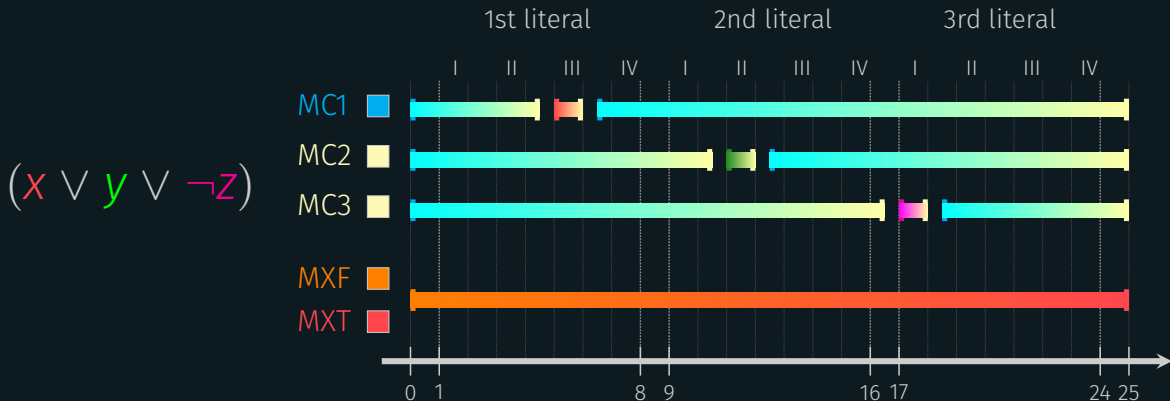
Interval Scheduling with Eligible Machines



Reduction from Exact (3,4)-SAT

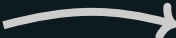


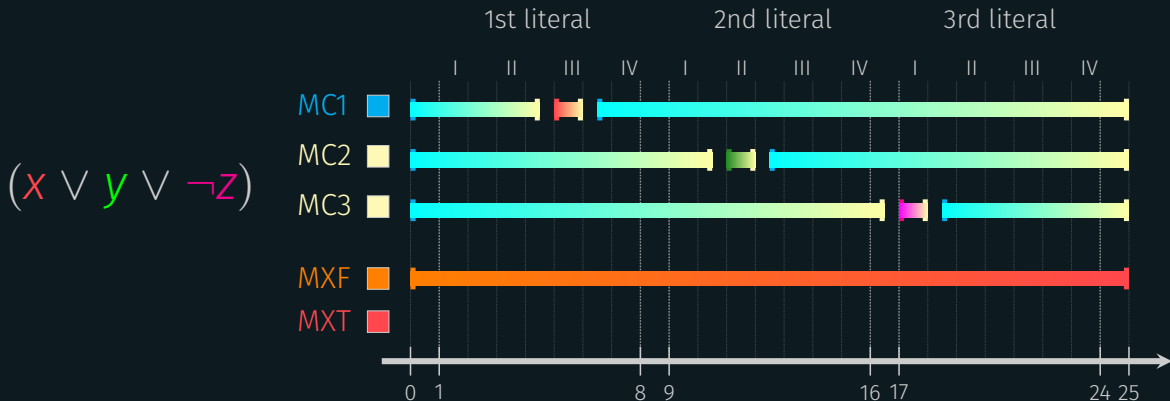
Exact (3,4)-SAT  Interval Scheduling with Eligible Machines



Reduction from Exact (3,4)-SAT

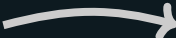


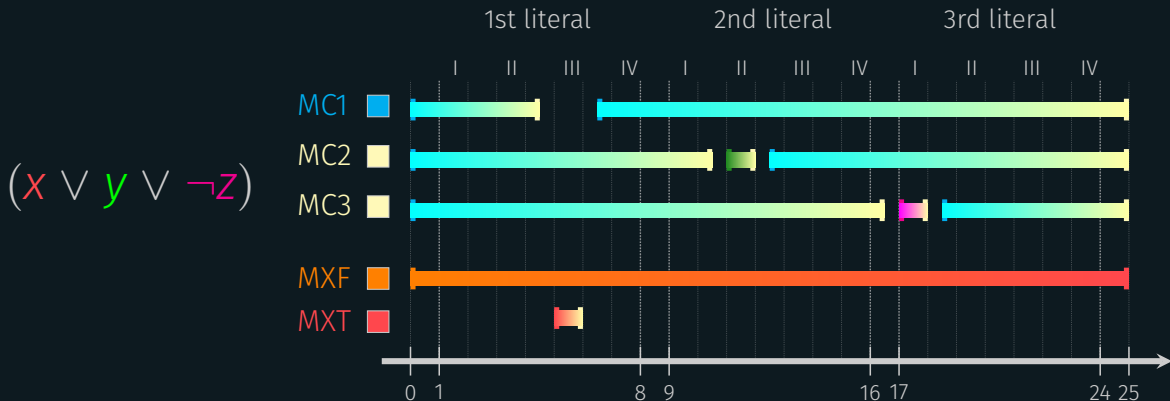
Exact (3,4)-SAT  Interval Scheduling with Eligible Machines



Reduction from Exact (3,4)-SAT

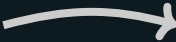


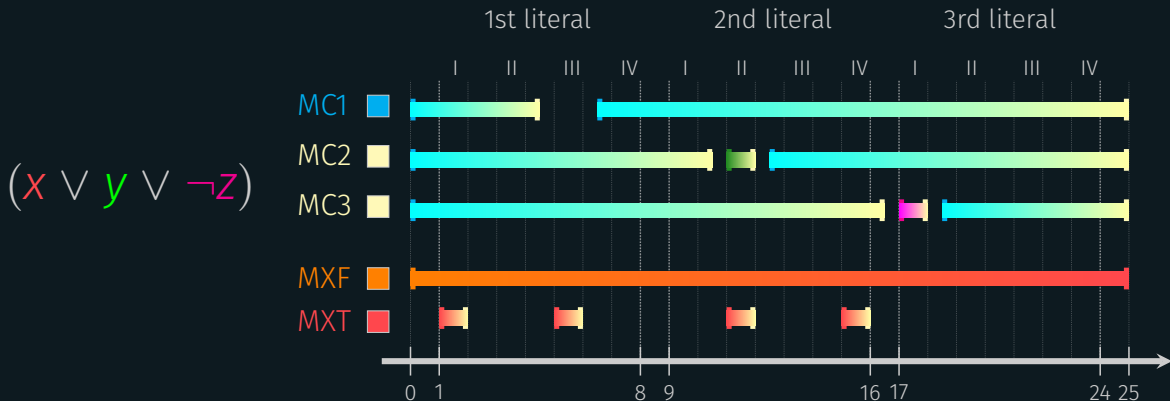
Exact (3,4)-SAT  Interval Scheduling with Eligible Machines



Reduction from Exact (3,4)-SAT



Exact (3,4)-SAT  Interval Scheduling with Eligible Machines



FPT w.r.t. $p_{\max} + m$

Theorem 3

The problem is FPT with respect to $p_{\max} + m$.

FPT w.r.t. $p_{\max} + m$



Theorem 3

The problem is FPT with respect to $p_{\max} + m$.

- Removal of jobs such that bounded number of jobs per time-step by $f(p_{\max}, m)$



Theorem 3

The problem is FPT with respect to $p_{\max} + m$.

- Removal of jobs such that bounded number of jobs per time-step by $f(p_{\max}, m)$
- Dynamic program over the time-steps

Theorem 3

The problem is FPT with respect to $p_{\max} + m$.

- Removal of jobs such that bounded number of jobs per time-step by $f(p_{\max}, m)$
- Dynamic program over the time-steps
- Key ideas:
 - At any time, at most m jobs can be processed
 - Some jobs are superior to others

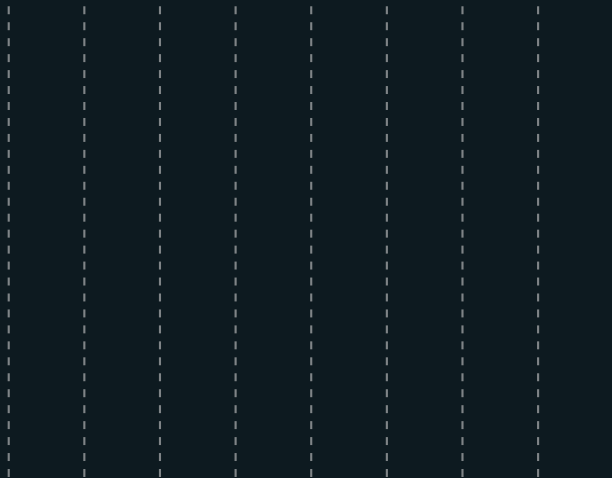
Data Reduction Procedure



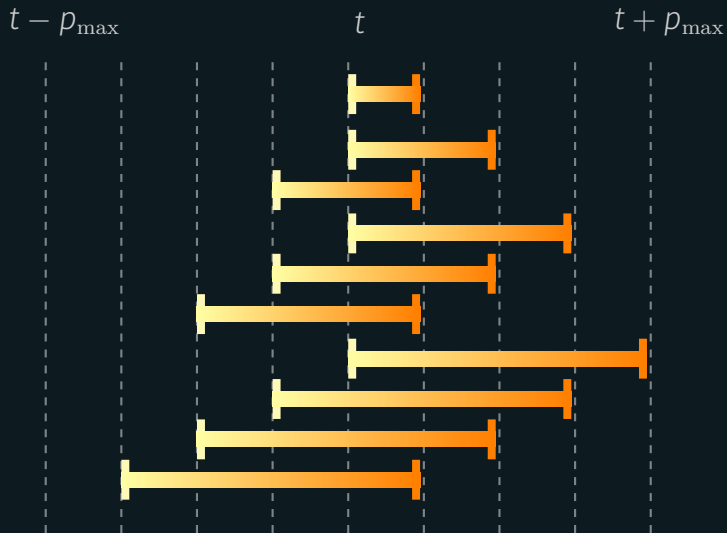
$$t - p_{\max}$$

$$t$$

$$t + p_{\max}$$



Data Reduction Procedure



For INTERVAL SCHEDULING ON ELIGIBLE MACHINES. there seems to exist no FPT algorithm with respect to m .

For INTERVAL SCHEDULING ON ELIGIBLE MACHINES. there seems to exist no FPT algorithm with respect to m .

- Does there exist FPT algorithm w.r.t. m when **uniform weights**?
- Does there exist FPT algorithm w.r.t. m when **uniform processing times**?

References

- [AS87] Esther M. Arkin and Ellen B. Silverberg. Scheduling jobs with fixed start and end times. *Discrete Applied Mathematics*, 18(1):1–8, 1987.
- [MvB18] Matthias Mnich and René van Bevern. Parameterized complexity of machine scheduling: 15 open problems. *Computers & Operations Research*, 100:254–261, 2018.